

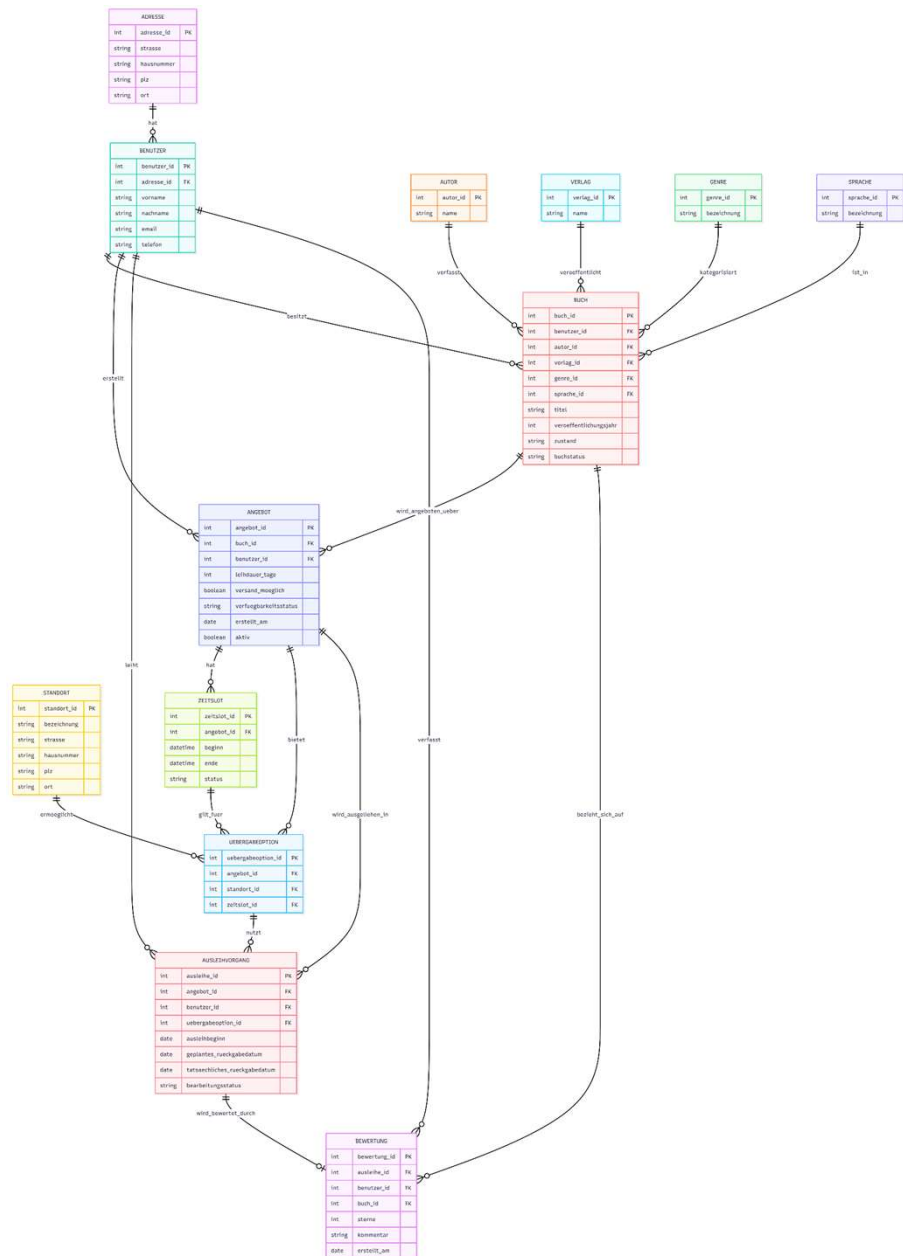
Buchtausch-App – Datenbankimplementierung (Phase 2)

Art der Arbeit:	Portfolio
Kursbezeichnung:	DLBDSPBDM01_D - Projekt: Data-Mart-Erstellung in SQL
Portfolioteil:	Portfolioteil 2 - Präsentations-PDF
Studiengang:	Bachelor Informatik (B.Sc.)
Datum:	18.08.2026
Name Verfasser:	Heiner Groenewold
Matrikelnummer:	92204285
Name Tutorin:	Anna Androvitsanea

Agenda

1. Rückblick ER-Modell
2. Entitäten mit SQL, Testfall & Ergebnis (13 Folien)
3. Komplexe Testabfragen

ER-Modell



ADRESSE

- Zweck: Speichert Wohnadressen der Benutzer
- SQL:

```
CREATE TABLE ADRESSE (
  adresse_id INTEGER PRIMARY KEY AUTOINCREMENT,
  strasse TEXT NOT NULL,
  hausnummer TEXT NOT NULL,
  plz TEXT NOT NULL,
  ort TEXT NOT NULL
);
```

- Testfall: SELECT * FROM ADRESSE;

DB Browser for SQLite | Geben Sie einen SQL-Ausdruck ein, um die Ergebnisse zu filtern (verwenden Sie Strg+)

	123 adresse_id	AZ strasse	AZ hausnummer	AZ plz	AZ ort
1	1	Bahnhofstraße	12	26382	Wilhelmshaven
2	2	Marktstraße	5	26389	Wilhelmshaven
3	3	Gökerstraße	88	26384	Wilhelmshaven
4	4	Peterstraße	21	26386	Wilhelmshaven
5	5	Freiligrathstraße	9	26385	Wilhelmshaven
6	6	Mühlenweg	3	26388	Wilhelmshaven
7	7	Rheinstraße	45	26382	Wilhelmshaven
8	8	Virchowstraße	17	26389	Wilhelmshaven
9	9	Adalbertstraße	60	26386	Wilhelmshaven
10	10	Kieler Straße	30	26384	Wilhelmshaven

BENUTZER

- Zweck: Stammdaten registrierter Nutzer

- SQL:

```
CREATE TABLE BENUTZER (
  benutzer_id INTEGER PRIMARY KEY AUTOINCREMENT,
  adresse_id INTEGER NOT NULL,
  vorname TEXT NOT NULL,
  nachname TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  telefon TEXT,
  FOREIGN KEY (adresse_id) REFERENCES ADRESSE(adresse_id)
);
```

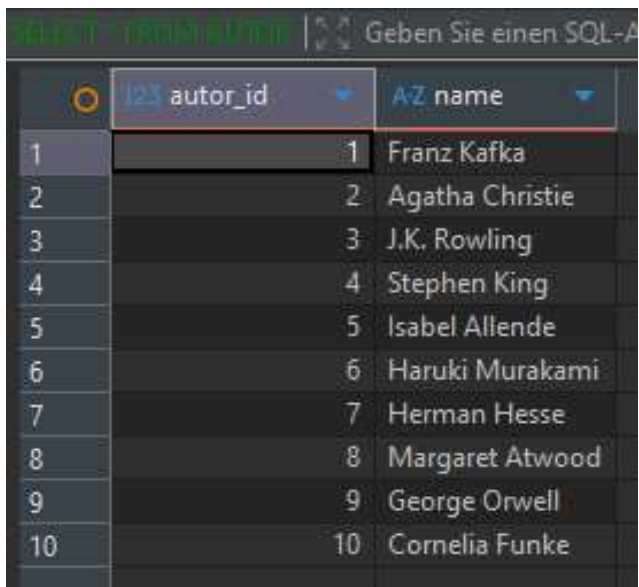
- Testfall: SELECT * FROM BENUTZER;

Geben Sie einen SQL-Ausdruck ein, um die Ergebnisse zu filtern (verwenden Sie Strg+ Leertaste).						
	benutzer_id	adresse_id	AZ vorname	AZ nachname	AZ email	AZ telefon
1	1	1	Anna	Schmidt	anna.schmidt@mail.de	04421-111111
2	2	2	Ben	Meyer	ben.meyer@mail.de	04421-222222
3	3	3	Clara	Fischer	clara.fischer@mail.de	04421-333333
4	4	4	David	Wagner	david.wagner@mail.de	04421-444444
5	5	5	Emma	Becker	emma.becker@mail.de	04421-555555
6	6	6	Finn	Hoffmann	finn.hoffmann@mail.de	04421-666666
7	7	7	Greta	Schulz	greta.schulz@mail.de	04421-777777
8	8	8	Hannes	Koch	hannes.koch@mail.de	04421-888888
9	9	9	Ida	Richter	ida.richter@mail.de	04421-999999
10	10	10	Jonas	Klein	jonas.klein@mail.de	04421-101010

AUTOR

- Zweck: Stammdaten der Buchautor:innen
- SQL:

```
CREATE TABLE AUTOR (  
    autor_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL  
);
```
- Testfall: `SELECT * FROM AUTOR;`



	123 autor_id	AZ name
1	1	Franz Kafka
2	2	Agatha Christie
3	3	J.K. Rowling
4	4	Stephen King
5	5	Isabel Allende
6	6	Haruki Murakami
7	7	Herman Hesse
8	8	Margaret Atwood
9	9	George Orwell
10	10	Cornelia Funke

VERLAG

- Zweck: Stammdaten der Verlage

- SQL:

```
CREATE TABLE VERLAG (
    verlag_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL
);
```

- Testfall: SELECT * FROM VERLAG;

SELECT * FROM VERLAG |

	123 verlag_id	AZ name
1	1	Rowohlt
2	2	Fischer Verlag
3	3	Suhrkamp
4	4	dtv
5	5	Piper Verlag
6	6	Diogenes
7	7	Hanser Verlag
8	8	Klett-Cotta
9	9	Ullstein
10	10	Beltz & Gelberg

GENRE

- Zweck: Stammdaten der Buchgenres

- SQL:

```
CREATE TABLE GENRE (  
    genre_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    bezeichnung TEXT NOT NULL  
);
```

- Testfall: `SELECT * FROM GENRE;`



Geben Sie einen SQL-Ausdruck

	genre_id	AZ bezeichnung
1	1	Krimi
2	2	Fantasy
3	3	Sachbuch
4	4	Roman
5	5	Biografie
6	6	Thriller
7	7	Kinderbuch
8	8	Kochbuch
9	9	Reisebericht
10	10	Science-Fiction

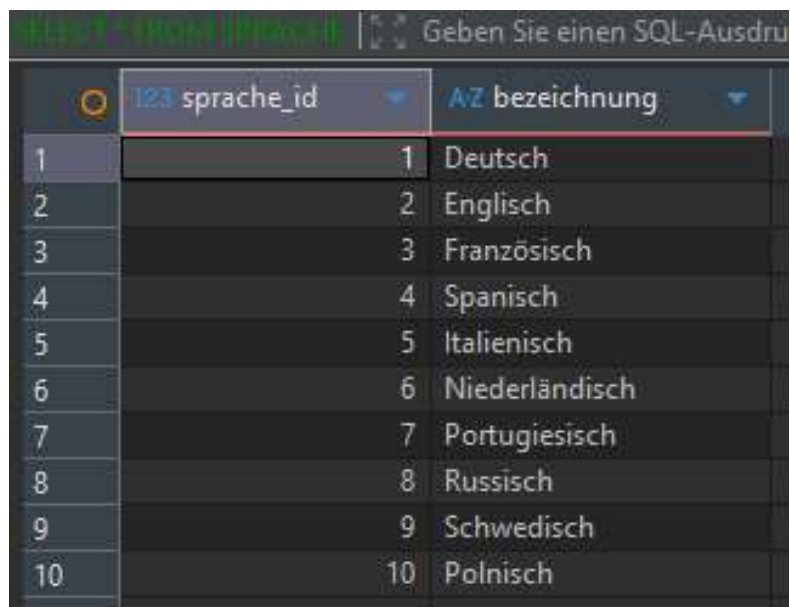
SPRACHE

- Zweck: Stammdaten der Buchsprachen

- SQL:

```
CREATE TABLE SPRACHE (  
    sprache_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    bezeichnung TEXT NOT NULL  
);
```

- Testfall: `SELECT * FROM SPRACHE;`



Geben Sie einen SQL-Ausdruck

	123 sprache_id	AZ bezeichnung
1	1	Deutsch
2	2	Englisch
3	3	Französisch
4	4	Spanisch
5	5	Italienisch
6	6	Niederländisch
7	7	Portugiesisch
8	8	Russisch
9	9	Schwedisch
10	10	Polnisch

STANDORT

- Zweck: Mögliche Übergabeorte
- SQL:

```
CREATE TABLE STANDORT (
    standort_id INTEGER PRIMARY KEY AUTOINCREMENT,
    bezeichnung TEXT NOT NULL,
    strasse TEXT NOT NULL,
    hausnummer TEXT NOT NULL,
    plz TEXT NOT NULL,
    ort TEXT NOT NULL
);
```

- Testfall: SELECT * FROM STANDORT;

Gegeben: **Übersichtstabelle** | **^** ***** Geben Sie einen SQL-Ausdruck ein, um die Ergebnisse zu filtern (verwenden Sie Strg+ Leertaste).

	standort_id	AZ bezeichnung	AZ strasse	AZ hausnummer	AZ plz	AZ ort
1	1	Cafe Lesestoff	Bahnhofstraße	10	26382	Wilhelmshaven
2	2	Stadtbibliothek	Rathausplatz	2	26382	Wilhelmshaven
3	3	Wochenmarkt Neuende	Neuender Straße	15	26386	Wilhelmshaven
4	4	Buchcafe Sud	Peterstraße	25	26386	Wilhelmshaven
5	5	Nordseepassage	Marktstraße	50	26389	Wilhelmshaven
6	6	Quartierstreff West	Mühlenweg	8	26388	Wilhelmshaven
7	7	Hafenpromenade	Rheinstraße	1	26382	Wilhelmshaven
8	8	Bürgerhaus Feddenwardergroden	Virchowstraße	20	26389	Wilhelmshaven
9	9	Kirchplatz Bant	Adalbertstraße	65	26386	Wilhelmshaven
10	10	Bahnhofsvorplatz	Kieler Straße	33	26384	Wilhelmshaven

BUCH

- Zweck: Konkretes Buchexemplar im Besitz eines Benutzers
- SQL

```
CREATE TABLE BUCH (
  buch_id      INTEGER PRIMARY KEY AUTOINCREMENT,
  benutzer_id  INTEGER NOT NULL,
  autor_id     INTEGER NOT NULL,
  verlag_id    INTEGER NOT NULL,
  genre_id     INTEGER NOT NULL,
  sprache_id   INTEGER NOT NULL,
  titel        TEXT NOT NULL,
  veroeffentlichungsjahr INTEGER,
  zustand      TEXT NOT NULL,
  buchstatus   TEXT NOT NULL CHECK (buchstatus IN
('verfuegbar','verliehen','inaktiv')),
  FOREIGN KEY (benutzer_id) REFERENCES BENUTZER(benutzer_id),
  FOREIGN KEY (autor_id)   REFERENCES AUTOR(autor_id),
  FOREIGN KEY (verlag_id)  REFERENCES VERLAG(verlag_id),
  FOREIGN KEY (genre_id)   REFERENCES GENRE(genre_id),
  FOREIGN KEY (sprache_id) REFERENCES SPRACHE(sprache_id)
);
```

- Testfall:

```
SELECT b.buch_id, b.titel, a.name AS autor, g.bezeichnung AS genre,
s.bezeichnung AS sprache, b.zustand, b.buchstatus
FROM BUCH b
JOIN AUTOR a ON b.autor_id = a.autor_id
JOIN GENRE g ON b.genre_id = g.genre_id
JOIN SPRACHE s ON b.sprache_id = s.sprache_id;
```

Geben Sie einen SQL-Ausdruck ein, um die Ergebnisse zu filtern (verwenden Sie Strg+ Leertaste).

	buch_id	titel	autor	genre	sprache	zustand	buchstatus
1	1	Die Verwandlung	Franz Kafka	Roman	Deutsch	gut	verfuegbar
2	2	Mord im Orientespress	Agatha Christie	Krimi	Deutsch	sehr gut	verfuegbar
3	3	Harry Potter und der Stein der Weisen	J.K. Rowling	Fantasy	Englisch	gebraucht	verliehen
4	4	Es	Stephen King	Thriller	Deutsch	gut	verfuegbar
5	5	Das Geisterhaus	Isabel Allende	Roman	Deutsch	neuwertig	verfuegbar
6	6	Kafka am Strand	Haruki Murakami	Roman	Englisch	gut	verfuegbar
7	7	Der Steppenwolf	Herman Hesse	Roman	Deutsch	gebraucht	verfuegbar
8	8	Der Report der Magd	Margaret Atwood	Science-Fiction	Englisch	sehr gut	verliehen
9	9	1984	George Orwell	Science-Fiction	Deutsch	gut	verfuegbar
10	10	Tintenherz	Cornelia Funke	Kinderbuch	Deutsch	neuwertig	verfuegbar

ANGEBOT

- Zweck: Buch, das zur Ausleihe bereitgestellt wird

- SQL:

```
CREATE TABLE ANGEBOT (
  anbot_id      INTEGER PRIMARY KEY AUTOINCREMENT,
  buch_id       INTEGER NOT NULL,
  benutzer_id   INTEGER NOT NULL,
  leihdauer_tage INTEGER NOT NULL,
  versand_moeglich INTEGER NOT NULL CHECK
(versand_moeglich IN (0,1)),
  verfuegbarkeitsstatus TEXT NOT NULL CHECK
(verfuegbarkeitsstatus IN ('verfuegbar','verliehen','inaktiv')),
  erstellt_am   DATE NOT NULL,
  aktiv         INTEGER NOT NULL CHECK (aktiv IN (0,1)),
  FOREIGN KEY (buch_id) REFERENCES BUCH(buch_id),
  FOREIGN KEY (benutzer_id) REFERENCES
BENUTZER(benutzer_id)
);
```

- Testfall:

```
SELECT an.angebot_id, b.titel, ben.vorname || ' ' || ben.nachname
AS anbieter,
      an.leihdauer_tage, an.versand_moeglich
FROM ANGEBOT an
JOIN BUCH b ON an.buch_id = b.buch_id
JOIN BENUTZER ben ON an.benutzer_id = ben.benutzer_id
WHERE an.verfuegbarkeitsstatus = 'verfuegbar';
```

SELECT an.angebot_id, b.titel, ben.vorname || ' ' || ben.nachname AS anbieter, an.leihdauer_tage, an.versand_moeglich

	123 anbot_id	AZ titel	AZ anbieter	123 leihdauer_tage	123 versand_moeglich
1	1	Die Verwandlung	Anna Schmidt	14	0
2	2	Mord im Orientexpress	Ben Meyer	21	1
3	4	Es	David Wagner	10	1
4	5	Das Geisterhaus	Emma Becker	30	0
5	6	Kafka am Strand	Finn Hoffmann	14	1
6	7	Der Steppenwolf	Greta Schulz	21	0
7	9	1984	Ida Richter	10	0
8	10	Tintenherz	Jonas Klein	14	1

ZEITSLOT

- Zweck: Verfügbare Zeitfenster je Angebot
- SQL:

```
CREATE TABLE ZEITSLOT (
    zeitslot_id INTEGER PRIMARY KEY AUTOINCREMENT,
    anbot_id INTEGER NOT NULL,
    beginn DATETIME NOT NULL,
    ende DATETIME NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('frei','gebucht')),
    FOREIGN KEY (anbot_id) REFERENCES ANBOT(anbot_id)
);
```

- Testfall:

```
SELECT z.zeitslot_id, b.titel, z.beginn, z.ende, z.status
FROM ZEITSLOT z
JOIN ANBOT an ON z.anbot_id = an.anbot_id
JOIN BUCH b ON an.buch_id = b.buch_id;
```

	123 zeitslot_id	AZ titel	AZ beginn	AZ ende	AZ status
1	1	Die Verwandlung	2026-08-20 10:00	2026-08-20 12:00	frei
2	2	Mord im Orientexpress	2026-08-21 14:00	2026-08-21 16:00	gebucht
3	3	Harry Potter und der Stein der Weisen	2026-08-18 09:00	2026-08-18 11:00	gebucht
4	4	Es	2026-08-22 15:00	2026-08-22 17:00	frei
5	5	Das Geisterhaus	2026-08-23 10:00	2026-08-23 12:00	frei
6	6	Kafka am Strand	2026-08-24 13:00	2026-08-24 15:00	frei
7	7	Der Steppenwolf	2026-08-25 16:00	2026-08-25 18:00	frei
8	8	Der Report der Magd	2026-08-15 09:00	2026-08-15 11:00	gebucht
9	9	1984	2026-08-26 10:00	2026-08-26 12:00	frei
10	10	Tintenherz	2026-08-27 14:00	2026-08-27 16:00	frei

UEBERGABEOPTION

- Zweck: Verknüpft Angebot, Standort und Zeitslot
- SQL:

```
CREATE TABLE UEBERGABEOPTION (
    uebergabeoption_id INTEGER PRIMARY KEY AUTOINCREMENT,
    angebot_id    INTEGER NOT NULL,
    standort_id   INTEGER NOT NULL,
    zeitslot_id   INTEGER NOT NULL,
    FOREIGN KEY (angebot_id) REFERENCES
    ANGEBOT(angebot_id),
    FOREIGN KEY (standort_id) REFERENCES
    STANDORT(standort_id),
    FOREIGN KEY (zeitslot_id) REFERENCES ZEITSLOT(zeitslot_id)
);
```

- Testfall:

```
SELECT u.uebergabeoption_id, b.titel, st.bezeichnung AS standort,
       z.beginn, z.ende
FROM UEBERGABEOPTION u
JOIN ANGEBOT an ON u.angebot_id = an.angebot_id
JOIN BUCH b ON an.buch_id = b.buch_id
JOIN STANDORT st ON u.standort_id = st.standort_id
JOIN ZEITSLOT z ON u.zeitslot_id = z.zeitslot_id;
```

Geben Sie einen SQL-Ausdruck ein, um die Ergebnisse zu filtern (verwenden Sie Strg+ Leertaste)					
	uebergabeoption_id	titel	standort	beginn	ende
1	123	1 Die Verwandlung	Cafe Lesestoff	2026-08-20 10:00	2026-08-20 12:00
2		2 Mord im Orientexpress	Stadtbibliothek	2026-08-21 14:00	2026-08-21 16:00
3		3 Harry Potter und der Stein der Weisen	Wochenmarkt Neuende	2026-08-18 09:00	2026-08-18 11:00
4		4 Es	Buchcafe Süd	2026-08-22 15:00	2026-08-22 17:00
5		5 Das Geisterhaus	Nordseepassage	2026-08-23 10:00	2026-08-23 12:00
6		6 Kafka am Strand	Quartierstreff West	2026-08-24 13:00	2026-08-24 15:00
7		7 Der Steppenwolf	Hafenpromenade	2026-08-25 16:00	2026-08-25 18:00
8		8 Der Report der Magd	Bürgerhaus Feddenwardergroden	2026-08-15 09:00	2026-08-15 11:00
9		9 1984	Kirchplatz Bant	2026-08-26 10:00	2026-08-26 12:00
10		10 Tintenherz	Bahnhofsvorplatz	2026-08-27 14:00	2026-08-27 16:00

AUSLEIHVORGANG

- Zweck: Verknüpft Angebot, Benutzer und Übergabeoption
- SQL:

```
CREATE TABLE AUSLEIHVORGANG (
    ausleihe_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    angebot_id           INTEGER NOT NULL,
    benutzer_id          INTEGER NOT NULL,
    uebergabeoption_id   INTEGER NOT NULL,
    ausleihbeginn        DATE NOT NULL,
    geplantes_rueckgabedatum  DATE,
    tatsaechliches_rueckgabedatum DATE,
    bearbeitungsstatus   TEXT NOT NULL CHECK
(bearbeitungsstatus IN ('laufend','abgeschlossen')),
    FOREIGN KEY (angebot_id) REFERENCES
ANGEBOT(angebot_id),
    FOREIGN KEY (benutzer_id) REFERENCES
BENUTZER(benutzer_id),
    FOREIGN KEY (uebergabeoption_id) REFERENCES
UEBERGABEOPTION(uebergabeoption_id)
);
```

- Testfall:

```
SELECT av.ausleihe_id, b.titel, ben.vorname || ' ' || ben.nachname
AS ausleiher,
    av.ausleihbeginn, av.tatsaechliches_rueckgabedatum,
av.bearbeitungsstatus
FROM AUSLEIHVORGANG av
JOIN ANGEBOT an ON av.angebot_id = an.angebot_id
JOIN BUCH b ON an.buch_id = b.buch_id
JOIN BENUTZER ben ON av.benutzer_id = ben.benutzer_id;
```

Geben Sie einen SQL-Ausdruck ein, um die Ergebnisse zu filtern (verwenden Sie Strg+ Leertaste).						
	ausleihe_id	titel	ausleiher	ausleihbeginn	tatsaechliches_rueckgabedatum	bearbeitungsstatus
1	1	Harry Potter und der Stein der Weisen	David Wagner	2026-08-18	2026-08-24	abgeschlossen
2	2	Der Report der Magd	Anna Schmidt	2026-08-15	2026-08-21	abgeschlossen
3	3	Die Verwandlung	Ben Meyer	2026-07-01	2026-07-07	abgeschlossen
4	4	Mord im Orientexpress	Clara Fischer	2026-07-05	2026-07-14	abgeschlossen
5	5	Es	Emma Becker	2026-07-10	2026-07-16	abgeschlossen
6	6	Das Geisterhaus	Finn Hoffmann	2026-07-15	2026-07-24	abgeschlossen
7	7	Kafka am Strand	Greta Schulz	2026-07-20	2026-07-26	abgeschlossen
8	8	Der Steppenwolf	Hannes Koch	2026-07-25	2026-07-31	abgeschlossen
9	9	1984	Jonas Klein	2026-08-01	2026-08-07	abgeschlossen
10	10	Tintenherz	Anna Schmidt	2026-08-05	2026-08-11	abgeschlossen

BEWERTUNG

- Zweck: Verknüpft Benutzer, Ausleihvorgang und Buch

- SQL:

```
CREATE TABLE BEWERTUNG (
    bewertung_id INTEGER PRIMARY KEY AUTOINCREMENT,
    ausleihe_id INTEGER NOT NULL,
    benutzer_id INTEGER NOT NULL,
    buch_id INTEGER NOT NULL,
    sterne INTEGER NOT NULL CHECK (sterne BETWEEN 1 AND 5),
    kommentar TEXT,
    erstellt_am DATE NOT NULL,
    FOREIGN KEY (ausleihe_id) REFERENCES AUSLEIHVORGANG(ausleihe_id),
    FOREIGN KEY (benutzer_id) REFERENCES BENUTZER(benutzer_id),
    FOREIGN KEY (buch_id) REFERENCES BUCH(buch_id)
```

- Testfall:

```
SELECT be.bewertung_id, b.titel, ben.vorname || ' ' ||
ben.nachname AS bewertender,
    be.sterne, be.kommentar, be.erstellt_am
FROM BEWERTUNG be
JOIN BUCH b ON be.buch_id = b.buch_id
JOIN BENUTZER ben ON be.benutzer_id = ben.benutzer_id;
```

	103 bewertung_id	A1 titel	A2 bewertender	103 sterne	A2 kommentar	A2 erstellt_am
1	1	Harry Potter und der Stein der Weisen	David Wagner	5	Sehr guter Zustand, schnelle Übergabe.	2026-08-24
2	2	Der Report der Magd	Anna Schmidt	4	Buch war wie beschrieben.	2026-08-21
3	3	Die Verwandlung	Ben Meyer	5	Toller Klassiker, gerne wieder.	2026-07-07
4	4	Mord im Orientexpress	Clara Fischer	4	Spannend, kleine Gebrauchsspuren.	2026-07-14
5	5	Es	Emma Becker	3	Buch okay, Übergabe hat sich verzögert.	2026-07-16
6	6	Das Geisterhaus	Finn Hoffmann	5	Perfekter Zustand, freundlicher Kontakt.	2026-07-24
7	7	Kafka am Strand	Greta Schulz	4	Gute Erfahrung, Buch pünktlich zurückgegeben.	2026-07-26
8	8	Der Steppenwolf	Hannes Koch	5	Sehr empfehlenswert.	2026-07-31
9	9	1984	Jonas Klein	2	Buch stärker abgenutzt als angegeben.	2026-08-07
10	10	Tintenherz	Anna Schmidt	5	Reibungslose Ausleihe.	2026-08-11

Komplexe Testabfrage

Vollständige Ausleihhistorie mit Bewertung (JOIN über 4 Tabellen)

```
SELECT av.ausleihe_id, b.titel, be.sterne, be.kommentar,
       be.erstellt_am
FROM AUSLEIHVORGANG av
JOIN ANGEBOT an ON av.angebot_id = an.angebot_id
JOIN BUCH b ON an.buch_id = b.buch_id
JOIN BEWERTUNG be ON av.ausleihe_id = be.ausleihe_id;
```

	ausleihe_id	titel	sterne	kommentar	erstellt_am
1	1	Harry Potter und der St	5	Sehr guter Zustand, se	2026-08-24
2	2	Der Report der Magd	4	Buch war wie beschriel	2026-08-21
3	3	Die Verwandlung	5	Toller Klassiker, gerne v	2026-07-07
4	4	Mord im Orientexpress	4	Spannend, kleine Gebr	2026-07-14
5	5	Es	3	Buch okay, Übergabe b	2026-07-16
6	6	Das Geisterhaus	5	Perfekter Zustand, freu	2026-07-24
7	7	Kafka am Strand	4	Gute Erfahrung, Buch f	2026-07-26
8	8	Der Steppenwolf	5	Sehr empfehlenswert.	2026-07-31
9	9	1984	2	Buch stärker abgenutzt	2026-08-07
10	10	Tintenherz	5	Reibungslose Ausleihe.	2026-08-11